

SPRINT 4

Modo IA Câmera Jovi

Computational Thinking with Python

Documentação técnica da solução

Integrantes do grupo

Gustavo Bidin

Artur de Sá

Mateus Borges

Isadora Bradac

Setembro de 2026

Sumário

1. Descritivo do Projeto	3
1.1 Objetivos	3
1.2 Justificativa	3
1.3 Descrição das funcionalidades implementadas	3
1.4 Aspectos técnicos	4
1.5 API externa: Open-Meteo	5
1.6 Como executar	5
2. Justificativa das Alterações em Relação à Sprint Anterior	5
2.1 Visão geral	5
2.2 O que foi removido	5
2.3 O que foi modificado	6
2.4 O que foi adicionado	7
2.5 Resumo das mudanças	7

1. Descritivo do Projeto

1.1 Objetivos

Objetivo geral: desenvolver uma aplicação de terminal em Python, o **Modo IA Câmera Jovi**, que simula as funções de uma câmera inteligente voltada a estudantes, permitindo registrar, organizar, buscar e ajustar fotos de conteúdo de estudo (lousas, cadernos, slides) por matéria e por data.

Objetivos específicos:

- Agrupar as fotos por matéria para facilitar a revisão (Study AI).
- Ordenar as fotos cronologicamente de forma rápida (Fast Processing).
- Ajustar brilho e contraste de cada foto, com sugestão automática baseada no clima atual obtido de uma API externa (Adaptive Capture).
- Manter os dados salvos entre execuções por meio de persistência em arquivo JSON.
- Oferecer gerenciamento completo do acervo (cadastrar, buscar, listar, editar, apagar) e estatísticas de uso.

1.2 Justificativa

Estudantes costumam fotografar lousas e anotações durante as aulas, e essas imagens acabam misturadas na galeria do celular, o que dificulta a revisão antes de provas. Além disso, fotos tiradas com pouca luz ou em dias muito nublados ficam difíceis de ler. O projeto resolve os dois problemas: organiza o material por matéria e por data e orienta o ajuste de luz e contraste de acordo com a condição real do ambiente.

A persistência em JSON garante que o acervo não seja perdido ao fechar o programa, e o consumo de uma API pública de clima traz um dado real do ambiente para a decisão de ajuste. *Observação:* nesta versão as fotos são representadas por registros (nome, matéria, data, brilho e contraste); o programa não manipula arquivos de imagem, e sim simula as funcionalidades do modo de câmera.

1.3 Descrição das funcionalidades implementadas

O menu principal reúne as dez opções descritas a seguir, todas acessíveis por número.

Opção	Funcionalidade	Descrição
1	Cadastrar foto	Solicita nome e matéria (validados), registra a data/hora atual, define brilho e contraste iniciais em 50 e salva no JSON.
2	Study AI	Agrupa as fotos por matéria e exibe cada matéria com a quantidade de fotos e a lista de fotos.
3	Fast Processing	Ordena as fotos pela data de captura e as exibe em ordem cronológica numerada.
4	Adaptive Capture	Permite escolher uma foto e ajustar brilho e contraste (0 a 100) de forma manual ou por sugestão automática que consulta a API de clima. Se a API falhar, o programa cai automaticamente para o modo manual.
5	Buscar foto	Busca por trecho do nome ou da matéria, sem diferenciar maiúsculas de minúsculas.
6	Ver todas as fotos	Lista todas as fotos com nome, matéria, data, brilho e contraste.
7	Editar foto	Altera nome e/ou matéria; deixar o campo em branco mantém o valor atual.

Opção	Funcionalidade	Descrição
8	Apagar foto	Remove a foto escolhida após pedir confirmação ao usuário.
9	Estatísticas	Mostra total de fotos, total de matérias, matéria com mais fotos e médias de brilho e contraste.
10	Sair	Encerra o programa.

Após a execução de cada funcionalidade, o programa retorna ao menu principal, que só é encerrado pela opção 10.

1.4 Aspectos técnicos

Requisito da Sprint 4	Como foi atendido
Subrotinas com parâmetros e retorno	O sistema é dividido em 21 funções. Nenhuma depende de variável global: a lista de fotos é criada em <code>main()</code> e passada por parâmetro. Funções de apoio retornam valores (por exemplo, <code>validar_inteiro</code> , <code>buscar_clima</code> , <code>sugerir_ajuste</code> , <code>carregar_fotos</code>).
Validação de entrada e try/except	<code>validar_texto</code> rejeita entradas vazias. <code>validar_inteiro</code> trata <code>ValueError</code> e verifica o intervalo. Leitura do JSON trata <code>FileNotFoundError</code> , <code>JSONDecodeError</code> e registros com campos ausentes ou inválidos. A gravação trata <code>OSError</code> . A chamada à API trata falhas de rede, tempo esgotado e resposta inesperada. O menu trata <code>KeyboardInterrupt</code> e <code>EOFError</code> .
Persistência em arquivo	Os dados ficam em <code>fotos.json</code> . O arquivo é lido ao iniciar e regravado após cadastrar, ajustar, editar ou apagar. A data é convertida para texto ISO ao salvar e reconvertida para <code>datetime</code> ao carregar.
Consumo de API externa	API pública Open-Meteo (gratuita, sem chave), consultada com <code>urllib</code> da biblioteca padrão, com tempo limite de 5 segundos. Detalhes na seção 1.5.
Boas práticas	Sem estado global; constantes nomeadas (<code>ARQUIVO_JSON</code> , <code>FORMATO_DATA</code> , <code>API_CLIMA_URL</code>); funções auxiliares para eliminar repetição (<code>aguardar_enter</code> , <code>formatar_data</code> , <code>existem_fotos</code>); <code>main()</code> com <code>if __name__ == "__main__":</code> ; comentários de ENTRADA, PROCESSAMENTO e SAÍDA em cada função; nomes descritivos.
Interface intuitiva	Menu numerado, títulos por seção, mensagens de AVISO claras, confirmação antes de apagar, pausa com ENTER antes de voltar ao menu e mensagem inicial informando quantas fotos foram carregadas.
Integração com o restante da solução	O arquivo <code>fotos.json</code> , com estrutura simples e documentada abaixo, pode ser lido por outros módulos da solução. Além disso, o sistema integra-se à API Open-Meteo.

Estrutura do arquivo `fotos.json`:

```
[
  {
    "nome": "Lousa de Matematica",
    "matéria": "Matematica",
    "data": "2026-09-24T00:02:17.555006",
    "brilho": 80,
    "contraste": 70
  }
]
```

1.5 API externa: Open-Meteo

No Adaptive Capture, ao escolher a sugestão automática, a função `buscar_clima()` consulta o endpoint de previsão do Open-Meteo para as coordenadas de São Paulo, solicitando os campos `cloud_cover` (cobertura de nuvens em %) e `is_day` (1 para dia, 0 para noite). Em seguida, `sugerir_ajuste()` transforma esses valores em brilho e contraste:

Condição	Brilho	Contraste	Motivo
Noite	80	60	Compensa a pouca luz ambiente.
Dia com mais de 70% de nuvens	65	55	Luz mais fraca em dia muito nublado.
Dia com céu claro ou pouco nublado	50	50	Valores padrão.

Se não houver internet, a API estiver fora do ar ou a resposta vier em formato inesperado, o programa exibe um aviso e segue para o ajuste manual, sem travar.

1.6 Como executar

- Requisito: Python 3.10 ou superior (o código usa `match-case`). Não há bibliotecas externas para instalar.
- Execução: `python camera_jovi.py` na pasta do arquivo. O `fotos.json` é criado automaticamente no primeiro cadastro.
- A sugestão automática exige conexão com a internet; o restante do sistema funciona offline.

2. Justificativa das Alterações em Relação à Sprint Anterior

2.1 Visão geral

Na Sprint 3, o sistema já oferecia as 10 opções do menu, mas mantinha os dados apenas em memória (tudo era perdido ao fechar o programa), usava uma variável global compartilhada por todas as funções e não consumia nenhuma API. Na Sprint 4, **todas as funcionalidades do menu foram mantidas**, e as mudanças se concentraram em atender aos novos requisitos: persistência de dados, integração com API externa, melhor uso de funções com parâmetros e retorno, tratamento de exceções e boas práticas. O código passou de 293 para 476 linhas. O diff completo linha a linha está disponível no arquivo de diferenças entregue junto ao projeto.

2.2 O que foi removido

Nenhuma opção do menu foi removida. Foram retirados elementos de implementação que limitavam a qualidade do código:

Item removido	Justificativa
Variável global <code>fotos</code> usada diretamente dentro das funções	Funções dependentes de estado global são difíceis de testar e reaproveitar. Agora a lista é criada em <code>main()</code> e passada por parâmetro, como pede o enunciado ("parâmetros e retorno adequados").
Bloco do menu principal solto no escopo do módulo	Foi movido para <code>main()</code> , evitando execução automática ao importar o arquivo.
Repetição de <code>input("Pressione ENTER...")</code> e de <code>strftime(...)</code> em várias funções	Código duplicado foi substituído por <code>aguardar_enter()</code> e <code>formatar_data()</code> (princípio DRY).

Item removido	Justificativa
Perda de dados ao encerrar o programa	Deixou de existir com a persistência em JSON.

2.3 O que foi modificado

Item	Sprint 3	Sprint 4
Assinatura das funções	Sem parâmetros; liam e alteravam a lista global.	Recebem a lista de fotos por parâmetro (por exemplo, <code>cadastrar_foto(fotos)</code>).
Cadastrar, ajustar, editar e apagar	Alteravam apenas a lista em memória.	Após cada alteração chamam <code>salvar_fotos(fotos)</code> , gravando no JSON.
Adaptive Capture	Somente ajuste manual de brilho e contraste.	O usuário escolhe entre sugestão automática (API de clima) e ajuste manual. Se a API falhar, cai para o manual.
Apagar foto	Removia a foto imediatamente.	Pede confirmação (1 - Confirmar, 2 - Cancelar), pois a exclusão agora é gravada no arquivo.
Menu principal	Laço <code>while True</code> solto no módulo, sem tratamento de interrupção.	Dentro de <code>main()</code> : carrega os dados ao iniciar, informa quantas fotos foram carregadas e encerra de forma limpa em caso de Ctrl+C ou EOF.
Busca de fotos	Convertia o termo para minúsculas a cada comparação.	Converte o termo uma única vez antes do filtro.

2.4 O que foi adicionado

Item adicionado	Descrição e justificativa
Persistência em JSON (salvar_fotos, carregar_fotos)	Atende ao requisito de persistência. O carregamento valida cada registro e descarta os inválidos com aviso, em vez de travar.
Consumo de API externa (buscar_clima, sugerir_ajuste)	Atende ao requisito de API externa. Usa Open-Meteo e dá sentido real ao Adaptive Capture, alinhado à ideia do projeto.
Tratamento de exceções ampliado	FileNotFoundError, JSONDecodeError, OSError, KeyError, ValueError, TypeError, KeyboardInterrupt e EOFError, cada um com mensagem clara ao usuário.
Constantes (ARQUIVO_JSON, FORMATO_DATA, API_CLIMA_URL)	Evitam valores repetidos e facilitam a manutenção.
Funções auxiliares (aguardar_enter, formatar_data)	Eliminam código duplicado.
Função main() e if __name__ == "__main__"	Boa prática de organização; o arquivo pode ser importado sem executar o menu.
Confirmação antes de apagar	Evita perda acidental de dados agora que a exclusão é permanente no arquivo.
Novos imports (json, urllib.request, urllib.error)	Todos da biblioteca padrão; o projeto continua sem dependências externas.

2.5 Resumo das mudanças

Categoria	Quantidade / situação
Funcionalidades do menu mantidas	10 de 10
Funcionalidades removidas	0
Funcionalidades modificadas	4 com mudança de comportamento (Adaptive Capture, Apagar, Cadastrar e Editar); as demais só passaram a receber a lista por parâmetro
Novos recursos	Persistência em JSON e consumo de API externa
Funções no código	21 (na Sprint 3 eram 14, e o menu principal ficava fora de função)